



Name _____

Today's big idea

The computer can ask a **yes-or-no question** about the world, and do different things depending on the answer.

```
if <touching the player?> then
```

CONTROL

```
  ...do this...
```

```
else
```

CONTROL

```
  ...do that instead...
```

A question that can only be answered yes or no is called a **boolean**. Those blocks are **pointy hexagons**, and they only fit in pointy hexagon slots. Shape tells you what fits where.

Words to know

Word	What it means
condition	The yes/no question
boolean	An answer that can only be yes or no (true or false)
if / then	Do something only if the answer is yes
if / then / else	Do one thing if yes, a different thing if no
sensing	Blocks that let a sprite notice the world

Big one today: an `if` block checks **once**, at the moment it runs. If you want it to keep checking, it has to be **inside a loop**.

Class activity A – Traffic Light

```
when green flag clicked
```

EVENTS

forever	CONTROL						
<table border="1"> <tr> <td>if <touching [Red]?> then</td> <td></td> </tr> <tr> <td> <table border="1"> <tr> <td>say [STOP!]</td> <td>LOOKS</td> </tr> </table> </td> <td></td> </tr> </table>	if <touching [Red]?> then		<table border="1"> <tr> <td>say [STOP!]</td> <td>LOOKS</td> </tr> </table>	say [STOP!]	LOOKS		
if <touching [Red]?> then							
<table border="1"> <tr> <td>say [STOP!]</td> <td>LOOKS</td> </tr> </table>	say [STOP!]	LOOKS					
say [STOP!]	LOOKS						
<table border="1"> <tr> <td>else</td> <td>CONTROL</td> </tr> <tr> <td> <table border="1"> <tr> <td>say [GO!]</td> <td>LOOKS</td> </tr> </table> </td> <td></td> </tr> </table>	else	CONTROL	<table border="1"> <tr> <td>say [GO!]</td> <td>LOOKS</td> </tr> </table>	say [GO!]	LOOKS		
else	CONTROL						
<table border="1"> <tr> <td>say [GO!]</td> <td>LOOKS</td> </tr> </table>	say [GO!]	LOOKS					
say [GO!]	LOOKS						

Now drag your sprite in and out of the red one.

Class activity B – Bumper Sprite

when green flag clicked	EVENTS						
forever	CONTROL						
<table border="1"> <tr> <td>move 5 steps</td> <td>MOTION</td> </tr> </table>	move 5 steps	MOTION					
move 5 steps	MOTION						
<table border="1"> <tr> <td>if <touching [edge]?> then</td> <td>CONTROL</td> </tr> <tr> <td> <table border="1"> <tr> <td>turn 180 degrees</td> <td>MOTION</td> </tr> </table> </td> <td></td> </tr> </table>	if <touching [edge]?> then	CONTROL	<table border="1"> <tr> <td>turn 180 degrees</td> <td>MOTION</td> </tr> </table>	turn 180 degrees	MOTION		
if <touching [edge]?> then	CONTROL						
<table border="1"> <tr> <td>turn 180 degrees</td> <td>MOTION</td> </tr> </table>	turn 180 degrees	MOTION					
turn 180 degrees	MOTION						
<table border="1"> <tr> <td>wait 0.01 seconds</td> <td>CONTROL</td> </tr> </table>	wait 0.01 seconds	CONTROL					
wait 0.01 seconds	CONTROL						

Add a second sprite and make them bounce off each other too.

Your project – CATCHING!

Today your project becomes a real game you can actually play.

The new bit goes **inside the falling loop**:

repeat 20	CONTROL										
<table border="1"> <tr> <td>change y by -18</td> <td>MOTION</td> </tr> </table>	change y by -18	MOTION									
change y by -18	MOTION										
<table border="1"> <tr> <td>wait 0.02 seconds</td> <td>CONTROL</td> </tr> </table>	wait 0.02 seconds	CONTROL									
wait 0.02 seconds	CONTROL										
<table border="1"> <tr> <td>if <touching [Player]?> then</td> <td></td> </tr> <tr> <td> <table border="1"> <tr> <td>start sound [Pop]</td> <td>SOUND</td> </tr> </table> </td> <td></td> </tr> <tr> <td> <table border="1"> <tr> <td>go to x: (pick random -220 to 220) y: 170</td> <td>MOTION</td> </tr> </table> </td> <td></td> </tr> </table>	if <touching [Player]?> then		<table border="1"> <tr> <td>start sound [Pop]</td> <td>SOUND</td> </tr> </table>	start sound [Pop]	SOUND		<table border="1"> <tr> <td>go to x: (pick random -220 to 220) y: 170</td> <td>MOTION</td> </tr> </table>	go to x: (pick random -220 to 220) y: 170	MOTION		
if <touching [Player]?> then											
<table border="1"> <tr> <td>start sound [Pop]</td> <td>SOUND</td> </tr> </table>	start sound [Pop]	SOUND									
start sound [Pop]	SOUND										
<table border="1"> <tr> <td>go to x: (pick random -220 to 220) y: 170</td> <td>MOTION</td> </tr> </table>	go to x: (pick random -220 to 220) y: 170	MOTION									
go to x: (pick random -220 to 220) y: 170	MOTION										

- ☐ Catching works
 - ☐ A sound plays when you catch something
 - ☐ Playtest it — then swap seats and play your neighbour's!
 - ☐ Save it
-

Homework

1. **Catch feedback.** Make catching do something noticeable — a flash, a costume change, a different sound, a `say`.
 2. **Something you shouldn't catch.** Add a bad falling object (a bomb, a coconut, a cat). For now it just says "Ouch!" when it hits you. Next week it'll cost you points.
-

Bonus challenges

1. A sprite that says different things depending on which half of the stage it's on. (Hint: `if <(x position) > 0>`)
 2. A sprite that follows the mouse, but only while space is held down.
 3. Use `if` to stop your player leaving the stage — the thing we couldn't do in week 2!
 4. Use `and` to combine two conditions: only catch it if you're touching it **and** pressing space.
-

If something breaks

- **Catching never works?** The `if` isn't inside the loop.
 - **Works once, then never again?** The `if` is in the outer `forever` but not in the `repeat`.
 - **Object falls straight through you?** You need the `go to` inside the `if` to send it back up.
 - **Catching is really fussy?** Your object moves too far between checks. Make `change y by` smaller and the `repeat` number bigger.
-

Stuck? Put your hand up — that is what we are here for. And always save before you close the lid.